

SGP - CSP

Hamza Arharbi

January 2026

Contents

1	Introduction	2
2	Modélisation du problème Social Golfer	2
2.1	Paramètres du problème	2
2.2	Modélisation 1 : Vue par groupes	3
2.2.1	Variables de décision	3
2.2.2	Contraintes communes	3
2.2.3	Modèle de base	3
2.2.4	Bris de symétrie simple	4
2.2.5	Bris de symétries complet	4
2.2.6	Contraintes redondantes	5
2.2.7	Résultats et analyse de la modélisation par groupes	6
2.3	Modélisation 2 : Vue par joueurs	8
2.3.1	Principe	8
2.3.2	Variables de décision	8
2.3.3	Contraintes	8
2.3.4	Bris de symétries	8
2.3.5	Contraintes redondantes	9
2.3.6	Résultats et analyse de la modélisation par joueurs	10
3	Solveur ensembliste	11
3.1	Fonctionnement du solveur ensembliste	11
3.2	Algorithme du solveur	13
3.3	Variables ensemblistes	13
3.4	Contraintes ensemblistes	14
3.5	Heuristiques	15
3.5.1	Heuristique de choix de variable	15
3.5.2	Heuristique de choix de valeur	16
3.5.3	Nogoods	16
3.5.4	Construction de l'état à partir du chemin de recherche . .	17
3.5.5	Caching	17
3.6	Résultats expérimentaux	18

1 Introduction

Organiser un tournoi sportif où chaque participant doit rencontrer des adversaires différents chaque semaine est un défi qui dépasse la simple planification : c'est le cœur du problème du Social Golfer. Ce cas d'école de la satisfaction de contraintes (CSP) se distingue par ses nombreuses symétries et l'immensité de son espace de recherche, rendant sa résolution particulièrement complexe.

Ce rapport a pour objectif d'explorer les mécanismes de résolution de ce problème à travers deux approches complémentaires. Dans un premier temps, nous analyserons la modélisation formelle du problème en comparant deux visions distinctes : une approche centrée sur les groupes et une approche centrée sur les joueurs. Dans un second temps, nous détaillerons la conception et l'implémentation d'un solveur ensembliste dédié. Celui-ci repose sur la gestion de bornes (inférieures et supérieures) et la propagation de contraintes pour explorer efficacement l'espace combinatoire et converger vers une solution.

2 Modélisation du problème Social Golfer

Le problème du *Social Golfer* consiste à répartir un ensemble de golfeurs en groupes de taille fixe sur plusieurs semaines, de sorte que chaque golfeur joue exactement une fois par semaine et que deux golfeurs ne se rencontrent jamais plus d'une fois. Ce problème est un cas classique de problème de satisfaction de contraintes (CSP), présentant de nombreuses symétries et un espace de recherche très important.

Dans ce travail, nous proposons deux grandes familles de modélisation en MiniZinc :

- une modélisation **centrée sur les groupes** ;
- une modélisation alternative **centrée sur les joueurs**.

Pour chacune de ces modélisations, plusieurs variantes sont étudiées : modèle de base, bris de symétries et ajout de contraintes redondantes.

2.1 Paramètres du problème

Les paramètres communs à tous les modèles sont :

- q : nombre total de golfeurs ;
- g : nombre de groupes par semaine ;
- p : nombre de golfeurs par groupe ;
- w : nombre de semaines.

Une condition nécessaire à la faisabilité est imposée :

$$q = g \times p$$

Une borne théorique sur le nombre maximal de rencontres est également vérifiée :

$$w \times g \times \frac{p(p-1)}{2} \leq \frac{q(q-1)}{2}$$

Cette contrainte permet d'éliminer certaines instances irréalisables dès l'initialisation.

2.2 Modélisation 1 : Vue par groupes

2.2.1 Variables de décision

Dans cette première approche, les variables représentent directement les groupes formés chaque semaine.

$$\text{Groupes}[s, g] \subseteq \text{GOLFEURS}$$

où $\text{Groupes}[s, g]$ est l'ensemble des golfeurs affectés au groupe g de la semaine s . Les variables sont donc des variables ensemblistes.

2.2.2 Contraintes communes

Les contraintes suivantes sont présentes dans toutes les variantes de cette modélisation.

Taille des groupes Chaque groupe contient exactement p golfeurs :

$$\forall s, g, \quad |\text{Groupes}[s, g]| = p$$

Unicité par semaine Un golfeur ne peut appartenir qu'à un seul groupe par semaine :

$$\forall s, \quad \text{all_disjoint}(\{\text{Groupes}[s, g]\}_g)$$

Contrainte sociale Deux golfeurs ne peuvent jouer ensemble qu'une seule fois :

$$\forall s_1 < s_2, \forall g_1, g_2, \quad |\text{Groupes}[s_1, g_1] \cap \text{Groupes}[s_2, g_2]| \leq 1$$

2.2.3 Modèle de base

Le modèle `base.mzn` implémente uniquement les contraintes fondamentales. Bien que correct, ce modèle souffre d'un grand nombre de symétries :

- permutation des golfeurs ;
- permutation des groupes ;
- permutation des semaines.

Ces symétries entraînent une exploration redondante de l'espace de recherche.

2.2.4 Bris de symétrie simple

Le modèle `base_symetri_simple.mzn` introduit un premier bris de symétrie en fixant complètement la première semaine. L'idée est que, sans cette contrainte, la permutation des golfeurs et des groupes dans la première semaine générerait de multiples solutions équivalentes.

La contrainte introduite peut s'écrire mathématiquement comme suit :

$$\forall j \in \{1, \dots, g\}, \quad \text{Groupes}[1, j] = \{(j-1)p + 1, \dots, j \cdot p\}$$

Autrement dit, le premier groupe contient les joueurs 1 à p , le deuxième groupe $p + 1$ à $2p$, et ainsi de suite.

Effet : Cette contrainte :

- élimine toutes les solutions obtenues par permutation des golfeurs dans la première semaine ;
- élimine toutes les solutions obtenues par permutation des groupes dans la première semaine ;
- réduit considérablement l'espace de recherche du solveur sans enlever de solutions réalisables.

2.2.5 Bris de symétries complet

Le modèle `base_symetri.mzn` va plus loin en introduisant des contraintes de bris de symétries supplémentaires sur les semaines suivantes :

1. Ordre lexicographique des groupes dans chaque semaine Pour toute semaine $i \geq 2$, on impose un ordre lexicographique entre les groupes afin d'éviter les permutations équivalentes :

$$\forall i \in \{2, \dots, w\}, \quad \forall j \in \{1, \dots, g-1\}, \quad \text{Groupes}[i, j] \prec \text{Groupes}[i, j+1]$$

où $\prec$ représente l'ordre lexicographique des ensembles de golfeurs.

2. Ordre des semaines On impose également un ordre entre les semaines basé sur le premier groupe :

$$\forall i \in \{2, \dots, w-1\}, \quad \text{Groupes}[i, 1] \prec \text{Groupes}[i+1, 1]$$

Effet : Ces contraintes éliminent toutes les solutions obtenues par permutation :

- des groupes au sein d'une même semaine ;
- des semaines elles-mêmes.

2.2.6 Contraintes redondantes

Le modèle `base_redondant.mzn` introduit des contraintes supplémentaires pour améliorer la propagation, sans changer l'ensemble des solutions valides.

1. Matrice des rencontres On définit une matrice auxiliaire $Rencontres[i, j] \in \{0, 1\}$ qui indique si le joueur i a rencontré le joueur j au moins une fois.

$$Rencontres[i, j] = \begin{cases} 1 & \text{si } \exists s, g \text{ tel que } \{i, j\} \subseteq \text{Groupes}[s, g] \\ 0 & \text{sinon} \end{cases}$$

Cette contrainte est liée aux variables `Groupes` par un *channeling* :

$$\forall i < j, \quad Rencontres[i, j] = \text{bool2int} \left(\bigvee_{s, g} (\{i, j\} \subseteq \text{Groupes}[s, g]) \right)$$

2. Symétrie de la matrice et diagonale

$$Rencontres[i, j] = Rencontres[j, i], \quad \forall i, j$$

$$Rencontres[i, i] = 0, \quad \forall i$$

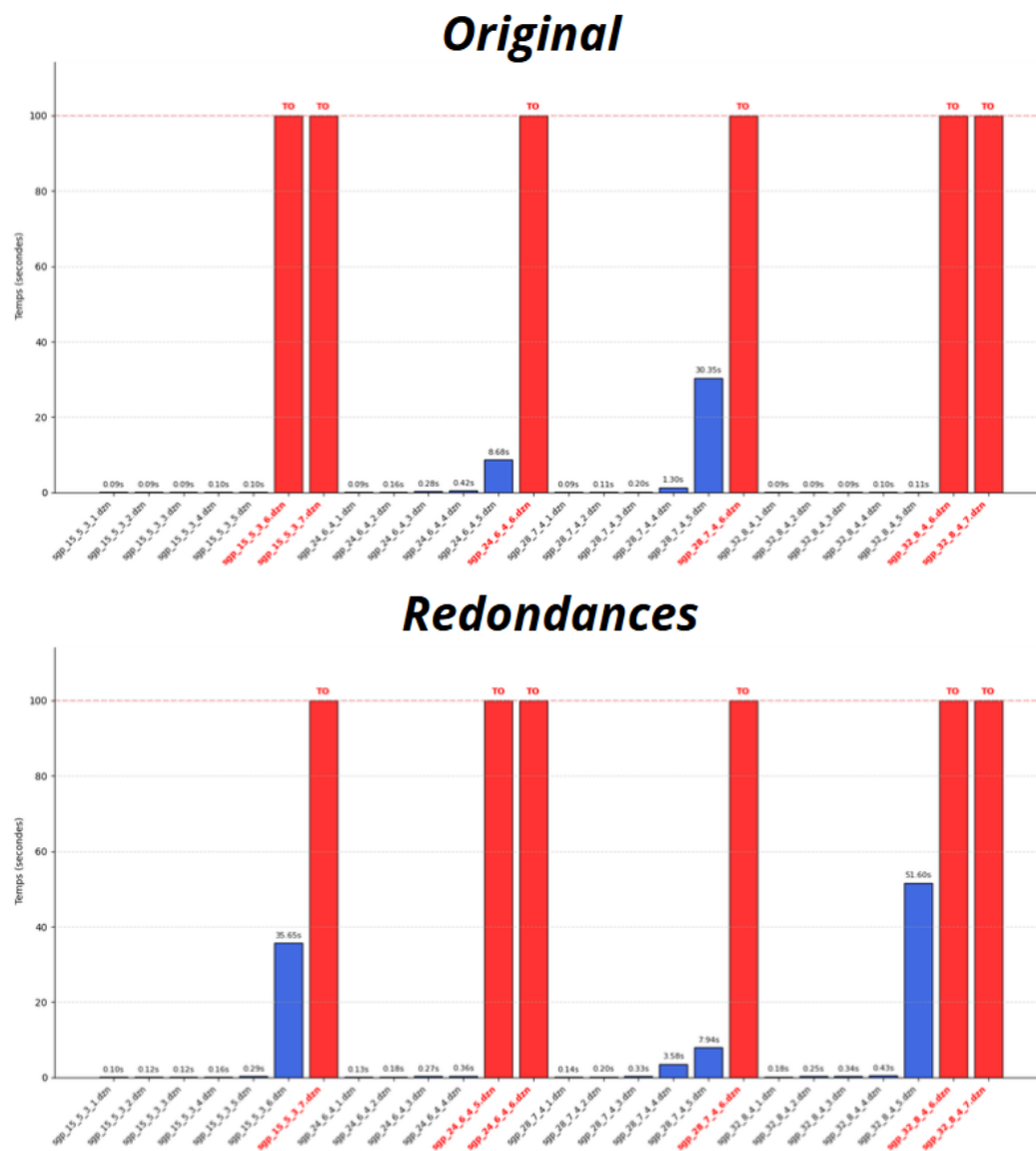
3. Contrainte sur le nombre exact de rencontres Chaque joueur doit rencontrer exactement $w(p - 1)$ adversaires distincts :

$$\forall i, \quad \sum_{j \in \text{GOLFEURS}} Rencontres[i, j] = w \cdot (p - 1)$$

Effet : Cette contrainte redondante permet :

- de détecter plus rapidement les conflits dans l'assignation des groupes ;
- d'améliorer la propagation des contraintes et de réduire l'espace de recherche ;
- de garantir que chaque joueur a rencontré exactement le nombre requis d'adversaires.

2.2.7 Résultats et analyse de la modélisation par groupes



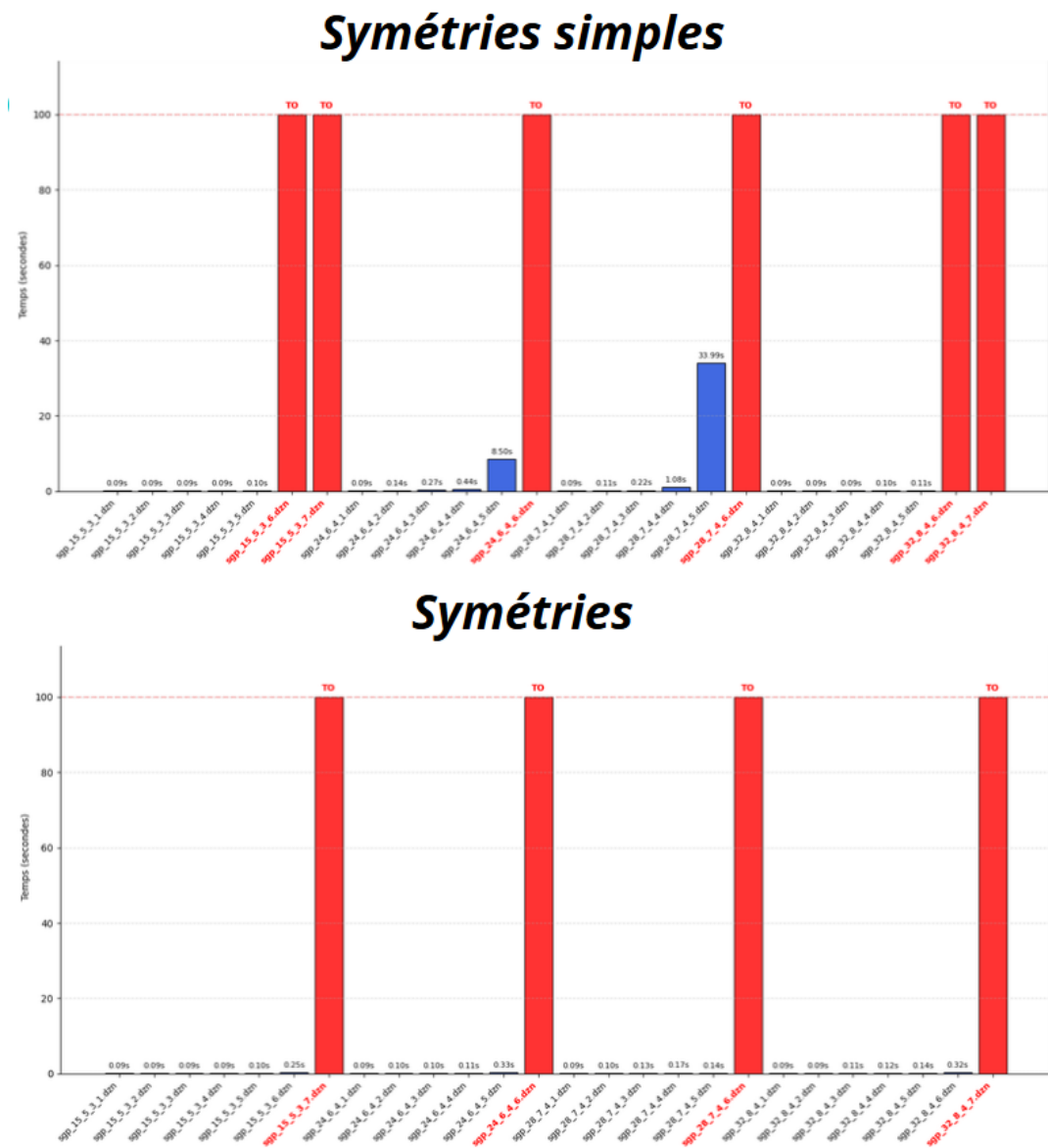


Figure 1: Visualisation des résultats des différentes implémentation sur plusieurs instances pour la modélisation par groupes

2.3 Modélisation 2 : Vue par joueurs

2.3.1 Principe

La seconde modélisation adopte un point de vue inversé. Chaque golfeur est associé aux groupes (appelés *slots*) auxquels il participe.

Un slot correspond à un couple (semaine, groupe) et est représenté par un entier.

2.3.2 Variables de décision

$$\text{Planning_Joueur}[j] \subseteq \text{SLOTS}$$

où $\text{Planning_Joueur}[j]$ représente l'ensemble des groupes auxquels participe le joueur j .

2.3.3 Contraintes

Participation hebdomadaire Chaque joueur joue exactement une fois par semaine :

$$|\text{Planning_Joueur}[j]| = w$$

Taille des groupes Chaque slot doit contenir exactement p joueurs :

$$\sum_j [k \in \text{Planning_Joueur}[j]] = p$$

Contrainte sociale Deux joueurs ne partagent au plus qu'un seul slot :

$$|\text{Planning_Joueur}[j_1] \cap \text{Planning_Joueur}[j_2]| \leq 1$$

2.3.4 Bris de symétries

Dans la modélisation par joueurs (`base2_redondant.mzn`), le bris de symétries est appliqué aux *slots* de chaque joueur pour réduire l'espace de recherche :

1. Fixation de la semaine 1 Chaque joueur j est assigné à un slot spécifique dans la première semaine, ce qui fixe complètement la configuration initiale :

$$\forall j \in \{1, \dots, q\}, \quad \left\lfloor \frac{j-1}{p} \right\rfloor + 1 \in \text{Planning_Joueur}[j]$$

2. Fixation du joueur 1 Le joueur 1 est utilisé comme étalon pour toutes les semaines afin de casser les symétries liées aux permutations de joueurs et de groupes dans les semaines suivantes :

$$\text{Planning_Joueur}[1] = \{k \cdot g + 1 \mid k = 0, \dots, w-1\}$$

Effet : Ces contraintes empêchent les solutions équivalentes obtenues par permutation des joueurs ou des slots, réduisant ainsi l'espace de recherche pour le solveur.

2.3.5 Contraintes redondantes

En complément du bris de symétries, des contraintes redondantes sont introduites pour améliorer la propagation et la faisabilité sociale.

Contrainte de rencontre globale On vérifie que chaque joueur rencontre exactement $w(p - 1)$ adversaires distincts via les slots :

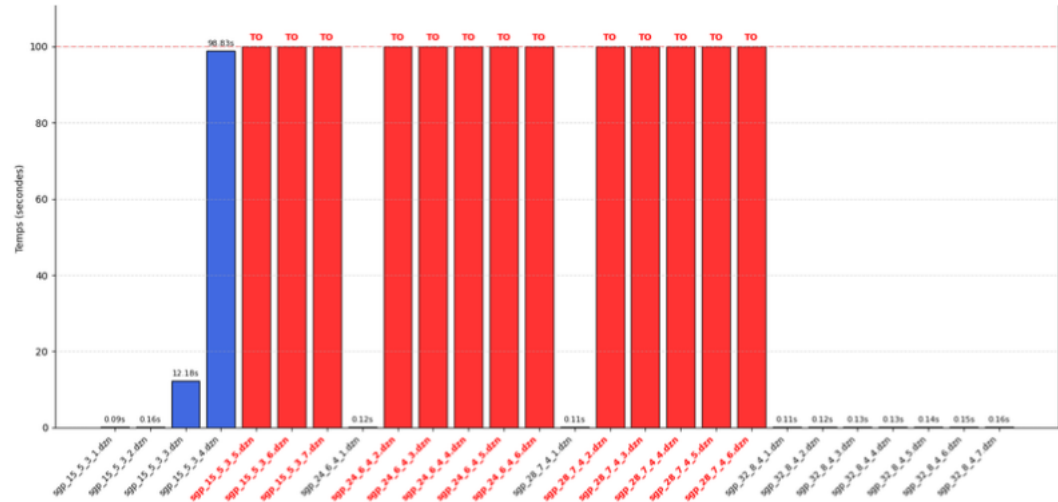
$$\forall j_1 \in GOLFEURS, \sum_{j_2 \in GOLFEURS, j_2 \neq j_1} |Planning_Joueur[j_1] \cap Planning_Joueur[j_2]| = w \cdot (p-1)$$

Effet : Cette contrainte :

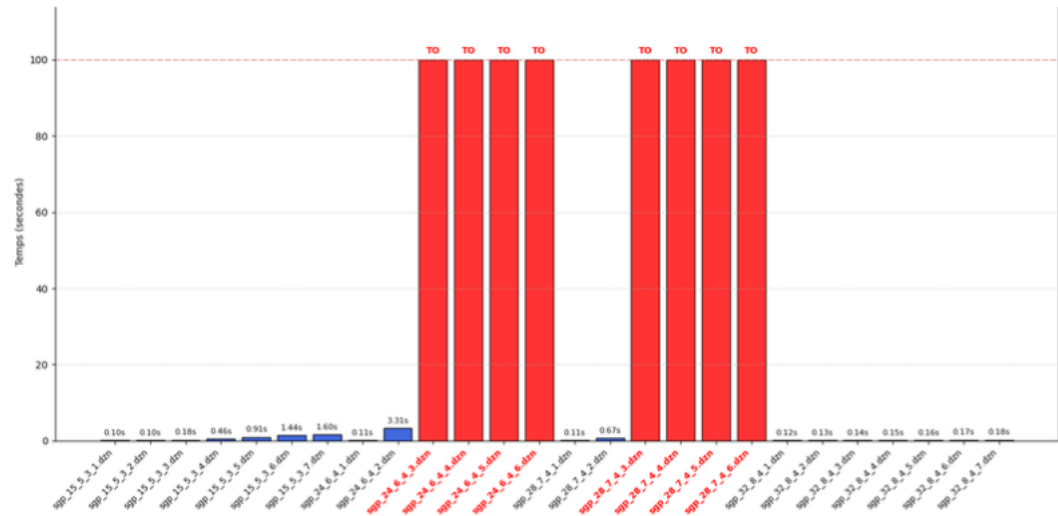
- accélère la propagation des contraintes dans le solveur ;
- réduit l'espace de recherche en détectant plus rapidement les conflits ;
- garantit que chaque joueur rencontre exactement le nombre requis d'adversaires distincts.

2.3.6 Résultats et analyse de la modélisation par joueurs

Original



Redondances



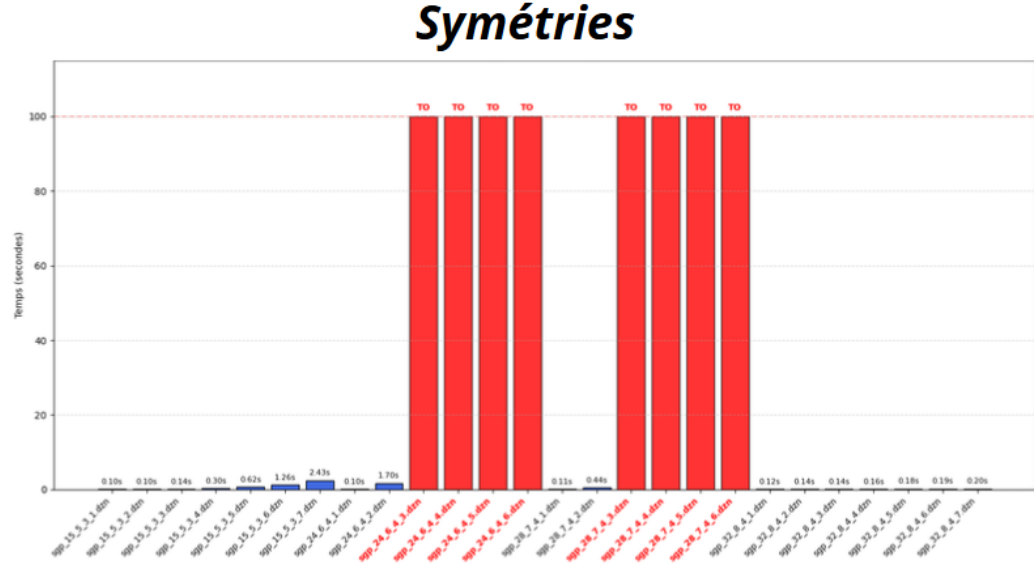


Figure 2: Visualisation des résultats des différentes implémentation sur plusieurs instances pour la modélisation par joueurs

3 Solveur ensembliste

3.1 Fonctionnement du solveur ensembliste

Le solveur repose sur un mécanisme de *backtracking* combiné à la propagation des contraintes et à l'apprentissage de *nogoods*. Le processus global peut se décrire comme suit :

1. **Vérification des nogoods** : Avant d'explorer un chemin, le solveur vérifie si celui-ci correspond à un chemin déjà identifié comme impossible (nogood). Si c'est le cas, il effectue un backtrack immédiat.
2. **Restart** : Si la profondeur maximale est atteinte, le solveur peut réinitialiser l'état des variables et relancer la recherche.
3. **Propagation des contraintes** : À chaque étape, les opérations appliquées sur les variables sont utilisées pour reconstruire l'état courant et propager toutes les contraintes jusqu'à stabilisation.
4. **Vérification de solution** : Si toutes les contraintes sont satisfaites par l'état courant, la solution est retournée.

5. **Choix de la variable et de la valeur** : Le solveur sélectionne une variable non déterminée selon une heuristique (par exemple MRV, nombre de contraintes, ou aléatoire), puis choisit une valeur à essayer pour cette variable.
6. **Backtracking** : Pour chaque valeur sélectionnée, le solveur tente d'ajouter cette valeur au lower bound de la variable. Si aucun chemin ne conduit à une solution, il retire cette valeur (upper bound) et explore d'autres options. Si toutes les possibilités sont épuisées, un *nogood* est appris et le solveur revient en arrière.

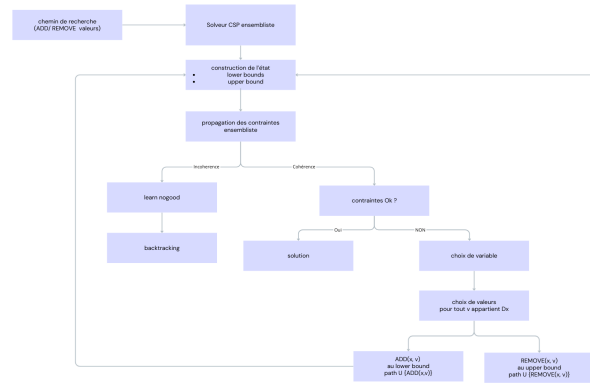


Figure 3: Architecture du solveur ensembliste

Ce mécanisme permet au solveur d'explorer efficacement l'espace combinatoire tout en évitant de répéter des chemins déjà identifiés comme impossibles, et en profitant de la propagation des contraintes pour réduire l'espace de recherche.

3.2 Algorithme du solveur

Algorithm 1 Solveur CSP ensembliste

```
1: function SOLVE
2:   while true do
3:     if Search renvoie RestartException then
4:       continue
5:     else
6:       return SEARCH( $\emptyset$ )
7:     end if
8:   end while
9: end function

10: function SEARCH(path)
11:   if ISNOGOOD(path) or RESTARTCRITERION then
12:     return None
13:   end if
14:   state  $\leftarrow$  BUILDSTATE(path)
15:   if state incohérent then
16:     LEARNNOGOOD(path)
17:     return None
18:   end if
19:   if ALLCONSTRAINTSSATISFIED(state) then
20:     return EXTRACTSOLUTION(state)
21:   end if
22:   (x, Dx)  $\leftarrow$  CHOOSEVARIABLE(state)
23:   for all v  $\in$  CHOOSEVALUES(Dx) do
24:     sol  $\leftarrow$  SEARCH(path  $\cup$  {ADD(x, v)})
25:     if sol  $\neq$  None then
26:       return sol
27:     end if
28:     sol  $\leftarrow$  SEARCH(path  $\cup$  {REMOVE(x, v)})
29:     if sol  $\neq$  None then
30:       return sol
31:     end if
32:   end for
33:   LEARNNOGOOD(path)
34:   return None
35: end function
```

3.3 Variables ensemblistes

Dans ce solveur, chaque variable est modélisée comme un **ensemble de valeurs possibles**, plutôt que comme une valeur unique. L'idée centrale est de maintenir

deux bornes pour chaque variable :

- **Lower bound (borne inférieure)** : ensemble des valeurs qui sont *déjà certaines* pour cette variable. Toute valeur présente dans le lower bound fait partie de la solution finale. Le lower bound ne peut jamais contenir de valeur absente du upper bound.
- **Upper bound (borne supérieure)** : ensemble des valeurs *encore possibles* pour cette variable. Toute valeur présente dans l'upper bound peut potentiellement faire partie de la solution. L'upper bound évolue au fur et à mesure de la propagation des contraintes et des choix effectués pendant la recherche.

Une variable est dite **déterminée** lorsque son lower bound et son upper bound *coïncident exactement*. À ce stade, la valeur de la variable est entièrement fixée et ne peut plus évoluer.

Le solveur fonctionne en **réduisant progressivement les domaines des variables** :

- lorsqu'une valeur est confirmée pour une variable, elle est ajoutée au lower bound ;
- lorsqu'une valeur est impossible, elle est retirée de l'upper bound.

Cette approche permet de **séparer les décisions certaines et possibles**, ce qui facilite la propagation des contraintes, le backtracking et l'apprentissage de nogoods.

Enfin, chaque variable conserve une **copie de ses bornes initiales**, ce qui permet de la *réinitialiser facilement* lors d'un restart, garantissant que l'état initial est toujours récupérable.

3.4 Contraintes ensemblistes

Le solveur utilise différents types de contraintes pour propager les informations et réduire les domaines des variables. Chaque contrainte agit sur un ou plusieurs ensembles de valeurs et permet de vérifier la cohérence des assignations.

- **Union** : cette contrainte assure que l'ensemble résultat est la *réunion* des ensembles des variables concernées.

Règle de filtrage :

- la borne supérieure du résultat est réduite pour contenir uniquement les valeurs possibles dans au moins une variable,
- la borne inférieure du résultat est augmentée pour inclure toutes les valeurs déjà certaines dans les variables.

- **Intersection** : cette contrainte garantit que l'ensemble résultat est l'*intersection* des ensembles des variables concernées.

Règle de filtrage :

- la borne inférieure du résultat est augmentée pour inclure uniquement les valeurs certaines dans toutes les variables,
 - la borne supérieure du résultat est réduite pour contenir uniquement les valeurs possibles dans toutes les variables.
- **CardinalityConstraint** : cette contrainte impose qu’une variable contienne exactement un nombre donné d’éléments.

Règle de filtrage :

- si le nombre de valeurs certaines atteint la cardinalité, toutes les autres valeurs possibles sont retirées de la borne supérieure,
 - si le nombre de valeurs restantes possibles est exactement ce qui manque pour atteindre la cardinalité, elles sont ajoutées à la borne inférieure.
- **IntersectionCardinalityConstraint** : cette contrainte limite le nombre d’éléments communs entre les ensembles de deux variables.

Règle de filtrage :

- toute valeur qui, si ajoutée à la borne inférieure d’une variable, ferait dépasser le maximum d’intersection autorisé est retirée de la borne supérieure,
- une incohérence est détectée immédiatement si l’intersection des bornes inférieures dépasse la limite.

Ces contraintes permettent au solveur de propager efficacement les informations, de réduire l’espace de recherche et de détecter rapidement les chemins impossibles, facilitant ainsi le backtracking et l’apprentissage de nogoods.

3.5 Heuristiques

3.5.1 Heuristique de choix de variable

Le solveur utilise différentes stratégies pour sélectionner la prochaine variable non déterminée sur laquelle effectuer une assignation. Le but de cette heuristique est de guider la recherche vers les variables les plus prometteuses, afin de réduire l’espace de recherche et d’augmenter l’efficacité du backtracking.

- **Variables non déterminées** : seules les variables dont le lower bound est différent de l’upper bound sont candidates au choix.
- **Stratégies de sélection :**
 - **Random** : une variable est choisie aléatoirement parmi les non-déterminées.

- **Minimum Remaining Values (MRV)** : la variable avec le plus petit nombre de valeurs encore possibles (upper bound moins lower bound) est sélectionnée, afin de traiter d’abord les variables les plus contraignantes.
- **Least Constraints** : la variable impliquée dans le moins de contraintes est sélectionnée, pour réduire le risque de propagation immédiate d’incohérences.
- **Most Constraints** : la variable impliquée dans le plus de contraintes est sélectionnée, pour maximiser l’effet de propagation et détecter tôt les conflits.
- **Randomisation et prévention des minima locaux** : Pour éviter de rester bloqué dans des minima locaux, le solveur peut sélectionner aléatoirement une variable parmi les K premières du classement selon la stratégie choisie après un certain nombre de choix consécutifs.
- **Restart automatique** : Si la profondeur de recherche maximale est atteinte, un *restart* est déclenché, réinitialisant les variables et l’état de recherche.

Cette heuristique permet de combiner exploitation (traiter les variables les plus critiques) et exploration (randomisation), améliorant l’efficacité du solveur ensembliste.

3.5.2 Heuristique de choix de valeur

Une fois la variable sélectionnée, le solveur choisit l’ordre dans lequel tester ses valeurs possibles. Les valeurs considérées sont celles présentes dans l’upper bound mais pas encore dans le lower bound.

- **Random** : les valeurs sont mélangées aléatoirement afin d’explorer l’espace de recherche sans biais.
- **Least Used / Minimum Occurrences** : les valeurs sont triées selon le nombre de fois où elles ont déjà été assignées précédemment à la variable. Les valeurs les moins utilisées sont essayées en premier, ce qui favorise la diversification de la recherche et réduit le risque de cycles.

Cette stratégie permet de combiner exploitation et exploration : - l’exploration par la randomisation aide à sortir des minima locaux, - l’exploitation via le comptage d’occurrences favorise l’utilisation des valeurs moins testées, équilibrant ainsi la recherche.

3.5.3 Nogoods

Les *nogoods* sont des chemins ou des assignations partiels connus comme étant impossibles. Ils permettent au solveur de **détecter rapidement les échecs récurrents** et d’éviter de réexplorer des sous-arbres inutiles.

- **Apprentissage des nogoods** : lorsqu'un chemin de recherche mène à un échec (aucune assignation ne peut satisfaire toutes les contraintes), ce chemin est transformé en nogood.
- **Vérification** : avant de continuer l'exploration d'un chemin, le solveur vérifie si le chemin courant correspond à un nogood existant. Si c'est le cas, la recherche est interrompue immédiatement, permettant un **backtracking anticipé**.
- **Objectif** : cette approche réduit significativement l'espace de recherche, car elle empêche la répétition des mêmes erreurs, et améliore l'efficacité du solveur ensembliste.

3.5.4 Construction de l'état à partir du chemin de recherche

Le solveur représente l'état courant des variables ensemblistes en appliquant successivement les *opérations* effectuées le long du chemin de recherche. Chaque opération correspond soit à l'ajout d'une valeur au *lower bound* d'une variable, soit à la suppression d'une valeur de son *upper bound*.

- **Application des opérations** : pour chaque opération du chemin, le solveur met à jour les bornes inférieures et supérieures des variables concernées. Cette mise à jour reflète toutes les décisions prises jusqu'à ce point dans l'exploration.
- **Propagation des contraintes** : après chaque mise à jour, toutes les contraintes sont appliquées de manière itérative jusqu'à ce qu'aucune nouvelle réduction ne soit possible. Cela permet de propager les implications des décisions prises et de détecter rapidement les incohérences.
- **Caching** : chaque état calculé est mis en cache. Si un même chemin est rencontré ultérieurement, le solveur peut réutiliser l'état déjà calculé, évitant ainsi des recalculs coûteux et accélérant la recherche.

Cette construction systématique de l'état à partir des opérations garantit que le solveur maintient toujours une représentation cohérente des variables et de leurs domaines, facilitant la propagation, le backtracking et l'apprentissage de nogoods.

3.5.5 Caching

Pour améliorer l'efficacité de la recherche, le solveur utilise un mécanisme de *caching* des états intermédiaires. Chaque chemin de recherche est représenté par la séquence d'opérations appliquées aux variables.

- Lorsqu'un chemin est exploré pour la première fois, l'état correspondant (i.e. les bornes inférieure et supérieure de toutes les variables) est calculé en appliquant toutes les opérations du chemin et en effectuant la propagation des contraintes jusqu'à stabilisation.

- L'état résultant est stocké dans un cache associant la représentation du chemin à l'état complet.
- Si le même chemin est rencontré à nouveau, le solveur peut récupérer directement l'état depuis le cache, évitant ainsi de recalculer les mises à jour des variables et la propagation des contraintes.
- Ce mécanisme réduit le temps de calcul, en particulier lorsque certains sous-arbres de l'espace de recherche sont visités plusieurs fois.

Ainsi, le caching permet d'optimiser le backtracking en minimisant les recalculs et en accélérant la détection des inconsistances.

3.6 Résultats expérimentaux

Les performances du solveur ensembliste ont été évaluées sur plusieurs instances du problème Social Golfer. Le tableau ci-dessous présente le temps de résolution pour chaque instance.

Semaines (W)	Groupes (G)	Joueurs par groupe (P)	Instance	Temps de résolution (s)
2	2	2	W2G2P2	0,053
3	3	2	W3G3P2	0,072
4	3	2	W4G3P2	0,085
4	3	3	W4G3P3	0,089
4	2	3	W4G2P3	—

Table 1: Temps de résolution des différentes instances du solveur ensembliste.